

DESPLEGAMENT D'APLICACIONS WEB

Práctica 10

Docker

ALUMNO

Stepan Andreev

Curso 2025 / 2026

Práctica 10 — Docker

Nota sobre el entorno

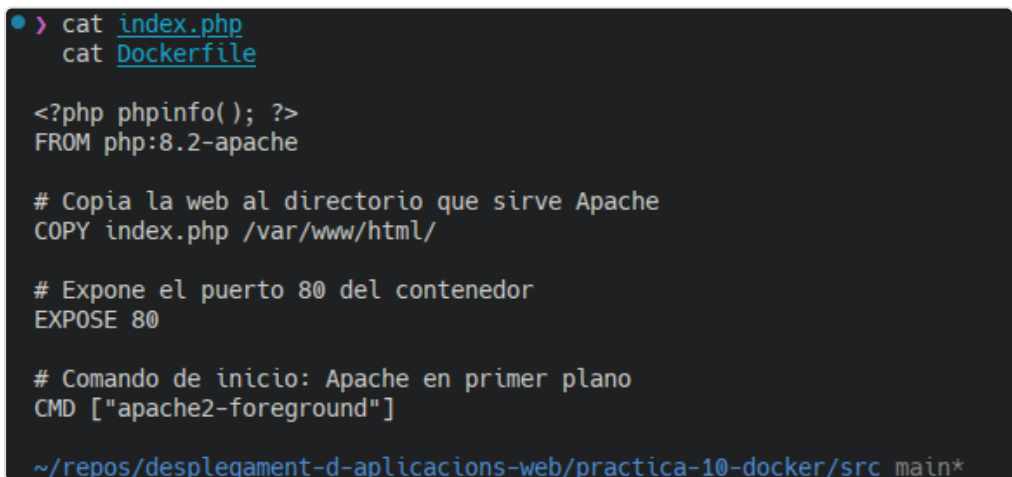
La práctica se ha realizado en local sobre **Arch Linux / CachyOS** con **Docker 29.5** y **Docker Compose v2**. Los datos elegidos (y anotados) para MySQL son: contraseña de root `rootpass`, base de datos `stepan_db` (formato `[nombre]_db`), usuario `stepan` con contraseña `stepanpass`. Se usan los puertos `8080` (web) y `8081` (phpMyAdmin).

Parte 1: Docker (Dockerfile)

1-2. `index.php` de prueba y `Dockerfile`

Se crea `src/index.php` con `<?php phpinfo(); ?>` y un `Dockerfile` basado en la imagen oficial **php:8.2-apache**, que copia `index.php` a `/var/www/html`, expone el puerto 80 y arranca Apache:

```
FROM php:8.2-apache
COPY index.php /var/www/html/
EXPOSE 80
CMD ["apache2-foreground"]
```



```
> cat index.php
cat Dockerfile

<?php phpinfo(); ?>
FROM php:8.2-apache

# Copia la web al directorio que sirve Apache
COPY index.php /var/www/html/

# Expone el puerto 80 del contenedor
EXPOSE 80

# Comando de inicio: Apache en primer plano
CMD ["apache2-foreground"]

~/repos/despliegament-d-aplicacions-web/practica-10-docker/src main*
```

`index.php` y `Dockerfile`

3. Construcción de la imagen

```
docker build -t miweb .
```

```

> docker build -t miweb .

6ba11ca2a037: Verifying Checksum
6ba11ca2a037: Download complete
6e52470c9616: Verifying Checksum
6e52470c9616: Download complete
f9752abfb65d: Download complete
74f9daa2de83: Verifying Checksum
74f9daa2de83: Download complete
4f4fb700ef54: Download complete
532ab1248593: Download complete
d6130d7d20c3: Verifying Checksum
d6130d7d20c3: Download complete
d6130d7d20c3: Pull complete
86b04aef2630: Pull complete
5eca6ea1eb30: Pull complete
58b2af9d79ca: Pull complete
7a07cb7b72a6: Pull complete
5dc7382b29fc: Pull complete
2911694b29da: Pull complete
532ab1248593: Pull complete
6ba11ca2a037: Pull complete
6e52470c9616: Pull complete
f9752abfb65d: Pull complete
74f9daa2de83: Pull complete
4f4fb700ef54: Pull complete
Digest: sha256:034cb2b91d74209744da4eec2696075ed97f52788668ffa68967fdd6ee6a306f
Status: Downloaded newer image for php:8.2-apache
----> 2d027385a1f8
Step 2/4 : COPY index.php /var/www/html/
----> 56eca4f56424
Step 3/4 : EXPOSE 80
----> Running in eeb9c42bcd48
----> Removed intermediate container eeb9c42bcd48
----> be2ab7da8cb4
Step 4/4 : CMD ["apache2-foreground"]
----> Running in b8cc65a6bd50
----> Removed intermediate container b8cc65a6bd50
----> 4104eb9e5f5f
Successfully built 4104eb9e5f5f
Successfully tagged miweb:latest

~/repos/despliegament-d-aplicacions-web/practica-10-docker/src main* 16s

```

docker build

4. Ejecución del contenedor

Se ejecuta enlazando el puerto 8080 del host al 80 del contenedor, con nombre `miweb` :

```
docker run -d -p 8080:80 --name miweb miweb
```

```

• > docker run -d -p 8080:80 --name miweb miweb

11b83a507a105c77049174ce165ea5d168709e6cc31b2450aec778264265b895

~/repos/despliegament-d-aplicacions-web/practica-10-docker/src main*

```

docker run

En `http://localhost:8080` se muestra la información de PHP (`phpinfo`):

PHP Version 8.2.31	
System	Linux 11b83a507a10 7.0.11-1-cachyos #1 SMP PREEMPT_DYNAMIC Thu, 04 Jun 2026 05:10:09 +0000 x86_64
Build Date	Jun 11 2026 00:33:04
Build System	Linux - Docker
Build Provider	https://github.com/docker-library/php
Configure Command	./configure --build=x86_64-linux-gnu --sysconfdir=/usr/local/etc --with-config-file-path=/usr/local/etc/php --with-config-file-scandir=/usr/local/etc/php/conf.d --enable-option-checking=fatal --with-mhash --with-pic --enable-mbstring --enable-mysqlnd --with-password-argon2 --with-sodium=shared --with-pdo-sqlite=/usr --with-sqlite=/usr --with-curl --with-iconv --with-openssl --with-readline --with-zlib --disable-phd --with-pcre --with-libdir=libx86_64-linux-gnu --disable-cgi --with-apcu
Server API	Apache 2.0 Handler
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/usr/local/etc/php
Loaded Configuration File	(none)
Scan this dir for additional .ini files	/usr/local/etc/php/conf.d
Additional .ini files parsed	/usr/local/etc/php/conf.d/docker-php-ext-opcache.ini, /usr/local/etc/php/conf.d/docker-php-ext-sodium.ini
PHP API	20220829
PHP Extension	20220829
Zend Extension	40220829
Zend Extension Build	API40220829.NTS
PHP Extension Build	API20220829.NTS
Debug Build	no
Thread Safety	disabled
Zend Signal Handling	enabled
Zend Memory Manager	enabled
Zend Multibyte Support	provided by mbstring
Zend Max Execution Timers	disabled
IPv6 Support	enabled
OTrace Support	disabled
Registered PHP Streams	https, ftps, compress.zlib, php, file, glob, data, http, ftp, phar
Registered Stream Socket Transports	tcp, udp, unix, udg, ssl, tls, tlsv1.0, tlsv1.1, tlsv1.2, tlsv1.3
Registered Stream Filters	zlib.*, convert.iconv.*, string.rot13, string.toupper, string.tolower, convert.*, consumed, dechunk

phpinfo en el navegador

5. Listado y parada del contenedor

```
docker ps
docker stop miweb
```

```
> docker ps
docker stop miweb

CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS
PORTS         miweb
0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
5f236f13413c   wordpress:latest                 "docker-entrypoint.s..." 3 months ago   Up 5 minutes
0.0.0.0:8000->80/tcp, [::]:8000->80/tcp
cker-compose-wordpress-1
cf5133b0ca89   mariadb:10.11.2                  "docker-entrypoint.s..." 3 months ago   Up 5 minutes
3306/tcp
cker-compose-db-1
32f00a873861   ghcr.io/open-webui/open-webui:main "bash start.sh"          5 months ago   Up 5 minutes (health
hy) 0.0.0.0:3000->8080/tcp, [::]:3000->8080/tcp
da622997c966   ollama/ollama:latest              "/bin/ollama serve"      5 months ago   Up 5 minutes
0.0.0.0:11434->11434/tcp, [::]:11434->11434/tcp
miweb

~/repos/despliegament-d-aplicacions-web/practica-10-docker/src main*
```

docker ps y stop

Parte 2: Docker-compose (MySQL)

6. `docker-compose.yml` con MySQL

Se define un `docker-compose.yml` con dos servicios: **web** (construido desde nuestro `Dockerfile`, puerto 8080:80) y **db** (imagen `mysql:8.0`).

Información añadida (variables de entorno de MySQL):

- `MYSQL_ROOT_PASSWORD: rootpass` — contraseña del usuario root de MySQL.
- `MYSQL_DATABASE: stepan_db` — base de datos creada automáticamente (formato `[nombre]_db`).
- `MYSQL_USER: stepan` y `MYSQL_PASSWORD: stepanpass` — usuario propio con su contraseña, con privilegios sobre `stepan_db`.

Además, `command: --general-log=1 --general-log-file=...` activa el registro general para poder ver las conexiones.

```
➤ cat docker-compose.yml

services:
  web:
    build: .
    ports:
      - "8080:80"
    depends_on:
      - db

  db:
    image: mysql:8.0
    command: --general-log=1 --general-log-file=/dev/stdout
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: stepan_db
      MYSQL_USER: stepan
      MYSQL_PASSWORD: stepanpass
    volumes:
      - ./datadir:/var/lib/mysql

~/repos/despliegament-d-aplicacions-web/practica-10-docker/src main*
```

`docker-compose.yml`

7. Volumen para persistencia (`datadir`)

Para mantener los datos de MySQL aunque se eliminen los contenedores, se define un **volumen de tipo bind** que enlaza la carpeta local `src/datadir` con `/var/lib/mysql` del contenedor:

```
volumes:
  - ./datadir:/var/lib/mysql
```

Tras el primer arranque, MySQL inicializa sus datos en `datadir` (se observa la carpeta de la base `stepan_db`):

```
➤ > ls -la datadir
drwxr-x--- - 999      14 jun 18:43 [] #innodb_redo
drwxr-x--- - 999      14 jun 18:43 [] #innodb_temp
drwxr-xr-x - 999      14 jun 18:43 [] .
drwxr-xr-x - castor909 14 jun 18:40 [] ..
drwxr-x--- - 999      14 jun 18:43 [] mysql
drwxr-x--- - 999      14 jun 18:43 [] performance_schema
drwxr-x--- - 999      14 jun 18:43 [] stepan_db
drwxr-x--- - 999      14 jun 18:43 [] sys
-rw-r----- 197k 999      14 jun 18:43 [] #ib_16384_0.dblwr
-rw-r----- 8,6M 999      14 jun 18:43 [] #ib_16384_1.dblwr
-rw-r----- 56 999      14 jun 18:43 [] auto.cnf
-rw-r----- 3,0M 999      14 jun 18:43 [] binlog.000001
-rw-r----- 157 999      14 jun 18:43 [] binlog.000002
-rw-r----- 32 999      14 jun 18:43 [] binlog.index
-rw-r----- 1,7k 999      14 jun 18:43 [] ca-key.pem
-rw-r--r-- 1,1k 999      14 jun 18:43 [] ca.pem
-rw-r--r-- 1,1k 999      14 jun 18:43 [] client-cert.pem
-rw-r----- 1,7k 999      14 jun 18:43 [] client-key.pem
-rw-r----- 5,7k 999      14 jun 18:43 [] ib_buffer_pool
-rw-r----- 13M 999      14 jun 18:43 [] ibdata1
-rw-r----- 13M 999      14 jun 18:43 [] ibtmp1
-rw-r----- 33M 999      14 jun 18:43 [] mysql.ibd
lrwxrwxrwx - 999      14 jun 18:43 [] mysql.sock -> /var/run/mysqld/mysqld.sock
-rw-r----- 1,7k 999      14 jun 18:43 [] private_key.pem
-rw-r--r-- 452 999      14 jun 18:43 [] public_key.pem
-rw-r--r-- 1,1k 999      14 jun 18:43 [] server-cert.pem
-rw-r----- 1,7k 999      14 jun 18:43 [] server-key.pem
-rw-r----- 17M 999      14 jun 18:43 [] undo_001
-rw-r----- 17M 999      14 jun 18:43 [] undo_002

~/repos/despliegament-d-aplicacions-web/practica-10-docker/src main*
```

Contenido de `datadir`

8. `db_test.php`, reconstrucción y verificación

Se crea `db_test.php`, que conecta con el servicio `db` usando los datos definidos. Se edita el `Dockerfile` para instalar la extensión **mysqli** (necesaria para conectar PHP con MySQL) y copiar `db_test.php`:

```
FROM php:8.2-apache
RUN docker-php-ext-install mysqli
COPY index.php /var/www/html/
COPY db_test.php /var/www/html/
EXPOSE 80
CMD ["apache2-foreground"]
```

```

• > cat db_test.php
    cat Dockerfile

<?php
$conn = new mysqli("db", "stepan", "stepanpass", "stepan_db");
if ($conn->connect_error) {
    die("Conexión fallida: " . $conn->connect_error);
}
echo "Conexión establecida con MySQL!";
?>
FROM php:8.2-apache

# Extensión mysqli para conectar con MySQL desde PHP
RUN docker-php-ext-install mysqli

# Copia la web al directorio que sirve Apache
COPY index.php /var/www/html/
COPY db_test.php /var/www/html/

# Expone el puerto 80 del contenedor
EXPOSE 80

# Comando de inicio: Apache en primer plano
CMD ["apache2-foreground"]

~/repos/despliegament-d-aplicacions-web/practica-10-docker/src main*

```

db_test.php y Dockerfile

Se arranca toda la pila reconstruyendo la imagen:

```
docker compose up -d --build
```

```

> docker compose up -d --build

Build complete.
Don't forget to run 'make test'.

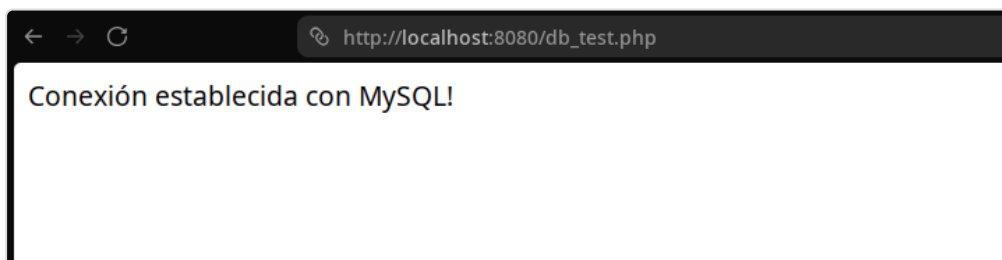
+ strip --strip-all modules/mysql.so
Installing shared extensions:      /usr/local/lib/php/extensions/no-debug-non-zts-20220829/
Installing header files:          /usr/local/include/php/
find . -name \*.gcno -o -name \*.gcda | xargs rm -f
find . -name \*.lo -o -name \*.o -o -name \*.dep | xargs rm -f
find . -name \*.la -o -name \*.a | xargs rm -f
find . -name \*.so | xargs rm -f
find . -name .libs -a -type d|xargs rm -rf
rm -f libphp.la      modules/* libs/*
rm -f ext/opcache/jit/Zend_jit_x86.c
rm -f ext/opcache/jit/Zend_jit_arm64.c
rm -f ext/opcache/minilua
----> Removed intermediate container 3aab908d5a5b
----> 6d744541c93f
Step 3/7 : COPY index.php /var/www/html/
----> 8a27796577c2
Step 4/7 : COPY db_test.php /var/www/html/
----> c03f6c446c84
Step 5/7 : EXPOSE 80
----> Running in 39cd10bdd6a6
----> Removed intermediate container 39cd10bdd6a6
----> f2013ebe20f2
Step 6/7 : CMD ["apache2-foreground"]
----> Running in ecbbae847ee3
----> Removed intermediate container ecbbae847ee3
----> 7b152061365e
Step 7/7 : LABEL com.docker.compose.image.builder=classic
----> Running in 9076a6897c78
----> Removed intermediate container 9076a6897c78
----> 39bdce37e3a9
[+] up 16/16 built 39bdce37e3a9
✓ Image mysql:8.0      Pulled
✓ Image src-web        Built
✓ Network src_default  Created
✓ Container src-db-1    Started
✓ Container src-web-1   Started

~/repos/despliegament-d-aplicacions-web/practica-10-docker/src main* 32s

```

docker compose up

En `http://localhost:8080/db_test.php` se confirma la conexión:



db_test.php en el navegador

Y en los logs de MySQL se ve la conexión realizada desde el navegador (`Connect stepan@... on stepan_db`):

```
docker exec src-db-1 tail -n 25 /var/lib/mysql/general.log
```



```

> docker exec src-db-1 tail -n 25 /var/lib/mysql/general.log

/usr/sbin/mysqld, Version: 8.0.46 (MySQL Community Server - GPL). started with:
Tcp port: 3306 Unix socket: /var/run/mysqld/mysqld.sock
Time Id Command Argument
2026-06-14T16:52:03.773117Z 0 Execute CREATE TABLE performance_schema.innodb_redo_log_files(
'FILE_ID' BIGINT NOT NULL COMMENT 'Id of the file.',
'FILE_NAME' VARCHAR(2000) NOT NULL COMMENT 'Path to the file.',
'START_LSN' BIGINT NOT NULL COMMENT 'LSN of the first block in the file.',
'END_LSN' BIGINT NOT NULL COMMENT 'LSN after the last block in the file.',
'SIZE_IN_BYTES' BIGINT NOT NULL COMMENT 'Size of the file (in bytes).',
'IS_FULL' TINYINT NOT NULL COMMENT '1 iff file has no free space inside.',
'CONSUMER_LEVEL' INT NOT NULL COMMENT 'All redo log consumers registered on smaller levels than this value, have
already consumed this file.'
)engine = 'performance_schema'
2026-06-14T16:53:01.399422Z 8 Connect stepan@172.24.0.3 on stepan_db using TCP/IP
2026-06-14T16:53:01.399911Z 8 Quit

~/repos/despliegament-d-aplicacions-web/practica-10-docker/src main*

```

Logs de MySQL con la conexión

Parte 3: phpMyAdmin y despliegue de la app

9. Añadir phpMyAdmin

Se detienen los contenedores:

```
docker compose down
```

```
> docker compose down
[+] down 3/3
✓ Container src-web-1 Removed 1.39s
✓ Container src-db-1 Removed 1.24s
✓ Network src_default Removed 0.15s
~/repos/despliegament-d-aplicacions-web/practica-10-docker/src main*
```

docker compose down

Se añade el servicio **phpmyadmin** al `docker-compose.yml`: imagen `phpmyadmin:latest`, puertos **8081:80**, y variables `PMA_HOST: db` (apunta al servicio MySQL) y `MYSQL_ROOT_PASSWORD: rootpass`.

```
> cat docker-compose.yml

services:
  web:
    build: .
    ports:
      - "8080:80"
    depends_on:
      - db

  db:
    image: mysql:8.0
    command: --general-log=1 --general-log-file=/var/lib/mysql/general.log
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: stepan_db
      MYSQL_USER: stepan
      MYSQL_PASSWORD: stepanpass
    volumes:
      - ./datadir:/var/lib/mysql

  phpmyadmin:
    image: phpmyadmin:latest
    ports:
      - "8081:80"
    environment:
      PMA_HOST: db
      MYSQL_ROOT_PASSWORD: rootpass
    depends_on:
      - db

~/repos/despliegament-d-aplicacions-web/practica-10-docker/src main*
```

compose con phpMyAdmin

```
docker compose up -d
```

```

➤ docker compose up -d

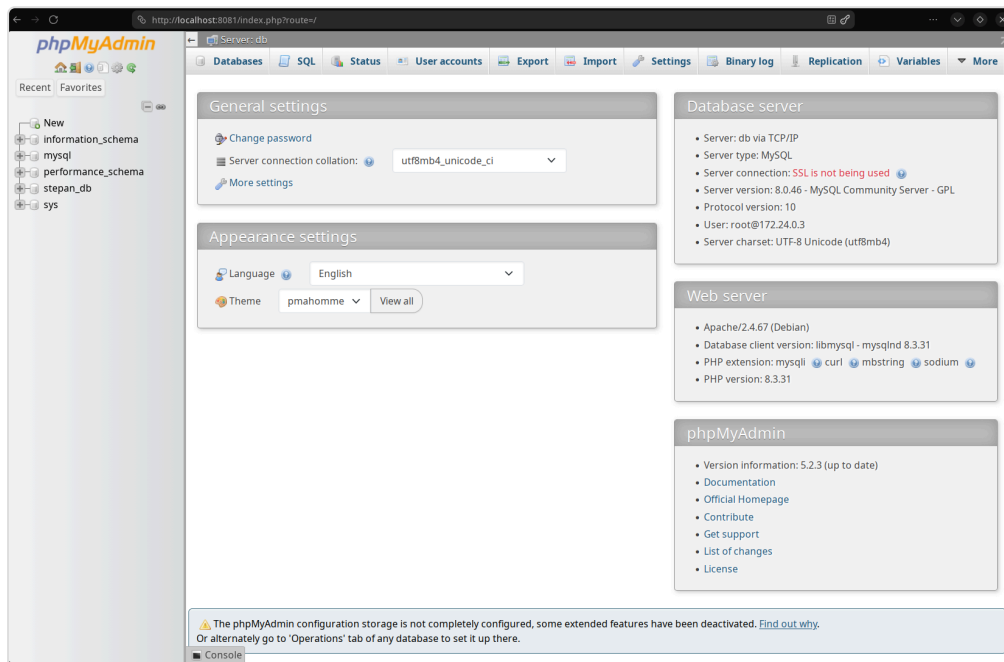
[+] up 26/26
✓ Image phpmyadmin:latest Pulled 15.3s
✓ Network src_default Created 0.0s
✓ Container src-db-1 Started 0.2s
✓ Container src-web-1 Started 0.3s
✓ Container src-phpmyadmin-1 Started 0.3s

~/repos/despliegament-d-aplicacions-web/practica-10-docker/src main* 15s

```

docker compose up

En `http://localhost:8081` se accede como **root** y se comprueba que aparece la base de datos `stepan_db` :



phpMyAdmin con `stepan_db`

10. Despliegue de la aplicación web (chat) y persistencia

Se detienen los contenedores (`docker compose down`):

```

➤ docker compose down

[+] down 4/4
✓ Container src-phpmyadmin-1 Removed 1.2s
✓ Container src-web-1 Removed 1.2s
✓ Container src-db-1 Removed 1.3s
✓ Network src_default Removed 0.1s

~/repos/despliegament-d-aplicacions-web/practica-10-docker/src main*

```

docker compose down

a) Se edita el `index.php` adjunto (chat) con los datos de conexión: servicio `db` , usuario `stepan` , contraseña `stepanpass` , base `stepan_db` :

```

> head -20 index.php

<?php
/*
Requiere crear la tabla en base de datos:
CREATE TABLE mensajes (
    id INT AUTO_INCREMENT PRIMARY KEY,
    usuario VARCHAR(50) NOT NULL,
    mensaje TEXT NOT NULL,
    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

*/

session_start();

// Configuración de la conexión a la base de datos
$host = "db";           // nombre del servicio MySQL en docker-compose
$user = "stepan";       // usuario definido en MYSQL_USER
$pass = "stepanpass";   // contraseña definida en MYSQL_PASSWORD
$db = "stepan_db";      // base de datos definida en MYSQL_DATABASE

~/repos/despliegament-d-aplicacions-web/practica-10-docker/src main*

```

Configuración de index.php

b) No es necesario modificar el `Dockerfile`: ya contiene `COPY index.php /var/www/html/`, por lo que al reconstruir toma el nuevo `index.php`. **c)** Tampoco hubo que cambiar `docker-compose.yml`.

d) Se reconstruye y arranca:

```
docker compose up -d --build
```

```

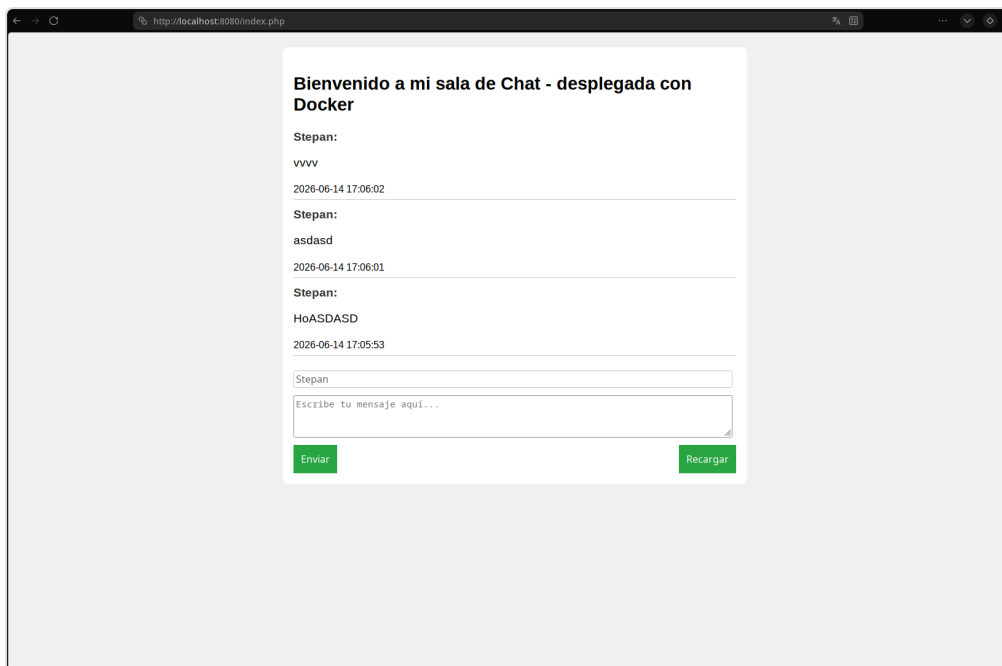
> sudo docker compose up -d --build
[sudo] password for castor909:
WARN[0000] Docker Compose requires buildx plugin to be installed
Sending build context to Docker daemon  5.565MB
Step 1/7 : FROM php:8.2-apache
--> 2d027385a1f8
Step 2/7 : RUN docker-php-ext-install mysqli
--> Using cache
--> 6d744541c93f
Step 3/7 : COPY index.php /var/www/html/
--> 6b2a6898db45
Step 4/7 : COPY db_test.php /var/www/html/
--> b48e0ebb766f
Step 5/7 : EXPOSE 80
--> Running in 5f7e3863f9e7
--> Removed intermediate container 5f7e3863f9e7
--> 890246f8af08
Step 6/7 : CMD ["apache2-foreground"]
--> Running in a1ca22f98f18
--> Removed intermediate container a1ca22f98f18
--> ab5ce65ad504
Step 7/7 : LABEL com.docker.compose.image.builder=classic
--> Running in 16a36ce9febb
--> Removed intermediate container 16a36ce9febb
--> a0776c9b1b09
Successfully built a0776c9b1b09
Successfully tagged src-web:latest
[+] up 5/5
✔ Image src-web           Built        6.3s
✔ Network src_default     Created      0.0s
✔ Container src-db-1      Started     0.2s
✔ Container src-web-1     Started     0.3s
✔ Container src-phpmyadmin-1 Started     0.3s

~/repos/despliegament-d-aplicacions-web/practica-10-docker/src main* 8s

```

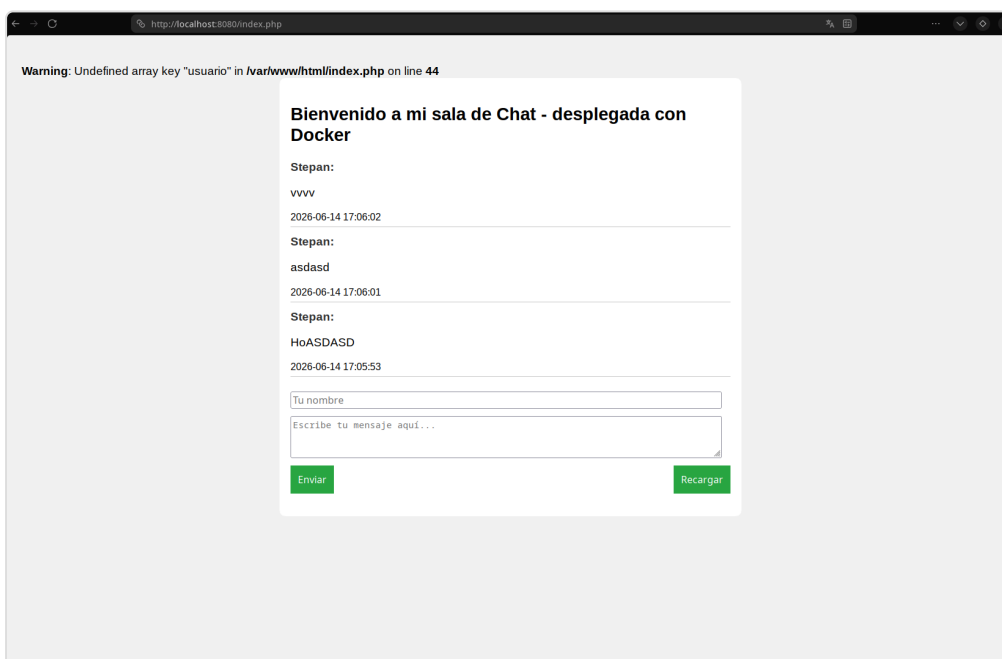
docker compose up -build

e) En `http://localhost:8080` funciona la aplicación de chat; se introducen varios



Aplicación de chat

f) Tras `docker compose down` y `docker compose up`, los mensajes **siguen existiendo** gracias al volumen `datadir`:



Mensajes persistentes tras reinicio

El aviso de PHP (`Undefined array key "usuario"`) proviene del código original adjunto y es inofensivo; no afecta a la persistencia.

Contenido de `src/datadir` (datos de MySQL persistidos):

```
➤ > ls -la datadir

drwxr-x--- - 999      14 jun 19:06 #innodb_redo
drwxr-x--- - 999      14 jun 19:06 #innodb_temp
drwxr-xr-x - 999      14 jun 19:06 .
drwxr-xr-x - castor909 14 jun 19:02 ..
drwxr-x--- - 999      14 jun 18:43 mysql
drwxr-x--- - 999      14 jun 18:43 performance_schema
drwxr-x--- - 999      14 jun 19:05 stepan_db
drwxr-x--- - 999      14 jun 18:43 sys
-rw-r----- 197k 999      14 jun 19:06 #ib_16384_0.dblwr
-rw-r----- 8,6M 999      14 jun 18:43 #ib_16384_1.dblwr
-rw-r----- 56 999      14 jun 18:43 auto.cnf
-rw-r----- 3,0M 999      14 jun 18:43 binlog.000001
-rw-r----- 180 999      14 jun 18:52 binlog.000002
-rw-r----- 180 999      14 jun 18:58 binlog.000003
-rw-r----- 180 999      14 jun 19:03 binlog.000004
-rw-r----- 2,6k 999      14 jun 19:06 binlog.000005
-rw-r----- 512 999      14 jun 19:06 binlog.000006
-rw-r----- 96 999      14 jun 19:06 binlog.index
-rw-r----- 1,7k 999      14 jun 18:43 ca-key.pem
-rw-r----- 1,1k 999      14 jun 18:43 ca.pem
-rw-r----- 1,1k 999      14 jun 18:43 client-cert.pem
-rw-r----- 1,7k 999      14 jun 18:43 client-key.pem
-rw-r----- 17k 999      14 jun 19:06 general.log
-rw-r----- 3,5k 999      14 jun 19:06 ib_buffer_pool
-rw-r----- 13M 999      14 jun 19:06 ibdata1
-rw-r----- 13M 999      14 jun 19:06 ibtmp1
-rw-r----- 33M 999      14 jun 19:06 mysql.ibd
lrwxrwxrwx - 999      14 jun 19:06 mysql.sock -> /var/run/mysqld/mysqld.sock
-rw-r----- 1,7k 999      14 jun 18:43 private_key.pem
-rw-r----- 452 999      14 jun 18:43 public_key.pem
-rw-r----- 1,1k 999      14 jun 18:43 server-cert.pem
-rw-r----- 1,7k 999      14 jun 18:43 server-key.pem
-rw-r----- 17M 999      14 jun 19:06 undo_001
-rw-r----- 17M 999      14 jun 19:06 undo_002

~/repos/despliegament-d-aplicacions-web/practica-10-docker/src main*
```

datadir tras reinicio

La tabla `mensajes` de `stepan_db` en phpMyAdmin conserva los registros:

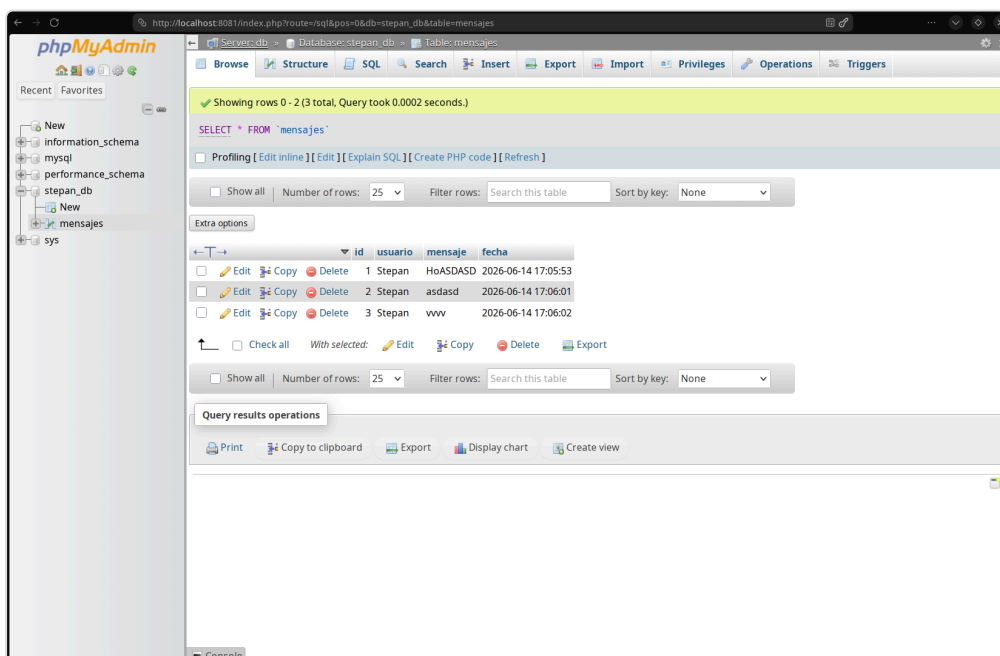


Tabla mensajes en phpMyAdmin

Conclusión

El resultado es una **pila LAMP desplegada con Docker**: Apache+PHP (servicio `web`), MySQL (servicio `db`) y phpMyAdmin (servicio `phpmyadmin`), orquestados con `docker-compose.yml` en una misma red. El volumen `datadir` garantiza la **persistencia** de los datos: los mensajes creados sobreviven al borrado y recreación de los contenedores.